

PARCIAL 1. PARADIGMAS DE PROGRAMACIÓN.

Andrés Sebastián Coral Vallejo.

1. PROBLEMA DE ORDENAMIENTO.

Para el ejercicio propuesto se planteo una solución de tres partes: Una implementación en paradigma declarativo utilizando lenguaje de programación Haskell, una implementación utilizando Python para el paradigma imperativo y por último una implementación en paradigma declarativo en Python. Esto se realizó de esa manera para facilitar mucho más la graficación del rendimiento en función del tiempo al utilizar un único entorno y también considerando que se nos pide evaluar la eficacia de los algoritmos y en términos generales del paradigma.

1.1. COMPARACIÓN DE CLARIDAD Y LEGIBILIDAD DEL CÓDIGO.

Haskell (declarativo)	Python imperativo (Quicksort manual)
Muy expresivo y conciso. La intención (ordenar con criterio compuesto) se ve casi inmediatamente.	Hay lógica de partición, índices, swaps y recursión explícita.
Hay menos “ruido”, en el sentido de que no hay gestión explícita de índices ni mutaciones.	Un programador de experiencia media lee fácilmente la implementación, pero hay más “ruido” que distrae de la intención principal (ordenar).
Para alguien que conoce Haskell, es muy legible; para quien no, la sintaxis y conceptos (composición, case, tipos) pueden requerir curva de aprendizaje.	Conclusión: El paradigma declarativo gana en legibilidad cuando el lector ya entiende el paradigma correctamente. Sin embargo, el imperativo expone los pasos concretos de forma que es útil para aprendizaje.



1.2. NIVEL DE EXPRESIVIDAD Y ABSTRACCIÓN.

- **Haskell:** Permitió un alto nivel de abstracción; usas funciones puras, compuestas. Puedes expresar el criterio de orden con una pequeña función `cmp` sin ser preocupante por la implementación del algoritmo de ordenamiento (Ya que la librería lo hace).

- ◆ **Python imperativo:** Permitió una menor abstracción, ya que se define el cómo explícitamente. En Python se puede lograr abstracción (Aplicando `sorted`), pero al implementarse manualmente se pierde esa ventaja.

- **Conclusión:** El paradigma funcional/declarativo favorece la expresividad y composición con menos código.

1.3. MANEJO DE ESTRUCTURAS DE DATOS. (MUTABILIDAD VS INMUTABILIDAD)

- **Haskell:** inmutabilidad por defecto. Las operaciones devuelven nuevas estructuras (o usan estructuras persistentes). Entre sus ventajas se tiene que existen menos efectos laterales, por lo que es más seguro para razonar y para concurrencia.

- ◆ **Python imperativo (Quicksort):** Muta la lista in-place, lo cual es eficiente en memoria. Esta mutabilidad facilita ciertas optimizaciones, pero complica razonar sobre estado compartido.

- **Conclusión:** La inmutabilidad mejora la seguridad y razonamiento; por otro lado, la mutabilidad puede mejorar rendimiento y uso de memoria en contextos concretos.

1.4. MANEJO DE ESTADO EN CADA PARADIGMA.

- **Haskell:** Sin estado mutable compartido, por lo que el estado es pasado explícitamente entre funciones o encapsulado, lo que reduce errores por efectos secundarios.

- ◆ **Python imperativo:** El estado (en este caso, la lista) se modifica en cada paso, lo que implica algunos riesgos como el aliasing, y otros errores por modificaciones inesperadas.

- **Conclusión:** Si el programa crece en complejidad y concurrencia, el enfoque funcional reduce clases enteras de errores, por lo que sería ideal tomarlo como alternativa.



1.5. FACILIDAD DE MANTENIMIENTO Y EXTENSIÓN DE CADA ENFOQUE.

Haskell (declarativo)	Python (imperativo)
Fácil de extender y componer, en este sentido, añadir otro criterio de ordenación suele hacer cambiar la función cmp.	Extender requiere tocar la lógica de algoritmo, lo que genera posibles efectos secundarios en el rendimiento o corrección.
Las pruebas y el razonamiento formal son más directos y se aplican con funciones puras.	En un contexto real, el aplicar sorted en Python ofrece una vía intermedia entre lo declarativo y mantenible.
Conclusión: El enfoque declarativo puro facilita mantenimiento; sin embargo, en la práctica usar librerías estándar en Python ofrece beneficios similares.	

1.6. EFICIENCIA DE CADA SOLUCIÓN. (CONSIDERANDO EL ALGORITMO Y EL LENGUAJE UTILIZADO)

Complejidad teórica de los algoritmos	Ambos enfoques pueden usar algoritmos del tipo $O(n \log n)$ en Haskell es sort y en Python es sorted/list.sort ambos usan algoritmos eficientes y estables.
---------------------------------------	--

Para encontrar la eficiencia se utilizaron los resultados empíricos a través de pruebas en Python, esto para comparar de manera más adecuada y en un mismo entorno las dos ejecuciones, de manera que se obtuvieron resultados como los vistos en la figura 1.

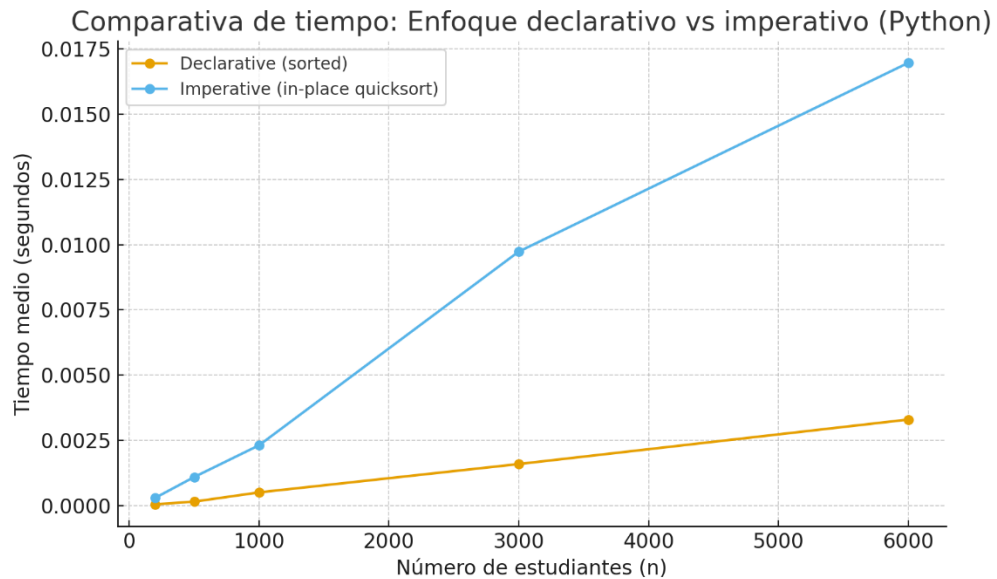


Figura 1. Grafica comparativa de tiempo para las implementaciones.

Con los resultados fue posible denotar que la implementación declarativa (sorted) fue consistentemente más rápida que la implementación imperativa (quicksort) esto al ser ejecutadas en Python (multiparadigma) para una mayor facilidad de comparación.

También se puede observar que, en los datos de prueba, la implementación imperativa era entre ~3–6× más lenta para los tamaños medidos.

Una acotación importante sobre Haskell es que la implementación usando sortBy es comparable en complejidad al algoritmo ($O(n \log n)$) por lo que se interpreta que, si se compila con optimizaciones y se usan tipos y estructuras adecuadas, su rendimiento puede ser muy competitivo respecto a otros lenguajes. Sin embargo, las implementaciones en la biblioteca estándar varían.

■ **Conclusión práctica:** Usar rutinas de librería optimizadas en el paradigma declarativo suele dar mejor rendimiento que reescribir algoritmos en Python puro. Entre lenguajes, C-implementations (como sorted en CPython) y compilación eficiente (GHC para Haskell) cambian mucho la comparación; por lo que si se tiene una cantidad de datos muy grande elegir la implementación correcta importa mucho.

2. GESTIÓN DE ESTUDIANTES.

Se solicitó un gestor en el Lenguaje C que nos permitiera esencialmente:

- Guardar registros de estudiantes (nombre, apellido, edad, id, conjunto de calificaciones).
- Usar malloc/free para asignación dinámica.
- Que cada registro ocupe solo la memoria necesaria (strings y arrays ajustados al tamaño real).
- Que haya compactación para reducir fragmentación y overhead.
- Usar struct optimizado/bitfields para campos pequeños (edad/id).
- Poder crear, borrar, listar, compactar, shrink-to-fit, y medir uso de memoria.

Para eso se realizó la implementación estando al pendiente de cumplir cada requerimiento solicitado por el profesor, en ese orden de ideas:

A) Registros y asignaciones dinámicas:

Estructura principal	<pre>typedef struct Student { char *first; char *last; unsigned int id:24; unsigned int age:7; size_t n_grades; double *grades; } Student;</pre>
----------------------	--

Por qué first y last son punteros a char, grades es puntero a double. Con esto podemos asignar exactamente lo que necesitamos en tiempo de ejecución.

Creación de un estudiante	Función: student_create(...) Hace malloc(sizeof(Student)), luego: s->first = strdup_malloc(first) → asigna strlen(first)+1 bytes. s->last = strdup_malloc(last) → asigna strlen(last)+1 bytes. si n_grades > 0: s->grades = malloc(n_grades * sizeof(double)) y copia las notas.
---------------------------	---

Así cada campo dinámico usa exactamente los bytes solicitados por el dato real; no hay arrays fijos (no char name[128]), por tanto se evita desperdicio de espacio por strings cortos.



B) Uso de malloc/free para crear y eliminar:

Añadir un estudiante	students_add_interactive() lee datos por consola, crea un arreglo temporal de double para la lectura de notas, llama a student_create() que copia las notas y strings, y libera el temporal. students_add() inserta el Student * en StudentArray (arreglo dinámico de punteros).
Eliminar un estudiante	La función students_delete() Si NO está compactado: libera s->first, s->last, s->grades, free(s) y compacta el array de punteros (shifting). Así free() se invoca exactamente sobre lo que se malloced. Si está compactado: realiza un desempaquetado seguro (clona cada estudiante en allocations individuales excepto el que se borra), libera el bloque compacto y deja la estructura en modo no-compactado. Es más costosa, pero garantiza consistencia y evita free() sobre punteros que apuntan dentro del bloque compacto.

C) Cada registro ocupa lo estrictamente necesario.

Los strings y arrays se asignan con strlen()+1 y n_grades * sizeof(double) respectivamente.

student_shrink_to_fit(Student *s)	Hace realloc para first a strlen+1, last a strlen+1, y grades a n_grades * sizeof(double). El resultado de realloc se usa únicamente si no es NULL, por lo que no se pierde el pointer original si falla. Así evitamos corrupción de datos si realloc falla.
students_ensure_capacity()	Asegura que el arreglo dinámico de punteros (arr->items) tenga capacidad razonable (doubling strategy) y así evita realocs constantes.

D) Estructuras optimizadas y bitfields.

id:24 y age:7 Son bitfields dentro de unsigned int	Reduce los bytes usados comparado con poner dos unsigned int completos para estos campos, cabe aclarar que en una plataforma típica de 32-bit, unsigned int = 4 bytes; por lo que usando campos se reduce un poco la representación por estudiante. id queda limitado a $< 2^{24}$ ($\approx 16.7M$), age a ≤ 127 por lo que si los datos de IDs o edades exceden ese límite habría que re-ajustar.
--	--

E) Compactación global.

Algoritmo paso a paso de students_compact_all(StudentArray *arr)	
1	Se calcula el tamaño total, entonces para cada estudiante sumo <code>align(struct) + sizeof(Student) + strlen(first)+1 + strlen(last)+1 + alineamiento para double + n_grades*sizeof(double)</code>. Se usa <code>align_up</code> para respetar alineamientos (importante para punteros, double, estructuras).
2	Se aplica <code>malloc(total)</code> dejando un único bloque contiguo.
3	Se recorren los estudiantes, copiando struct + strings + array de doubles dentro del bloque en posiciones calculadas; en el Student que se escribe dentro del bloque se ajusta first y last y grades a punteros que apuntan dentro del mismo bloque (manteniendo direcciones relativas).
4	Se crea un nuevo array <code>new_items</code> de <code>Student*</code> que apunta a esas estructuras dentro del bloque.
5	Se liberan las antiguas allocations (cada first/last/grades/Student) y <code>free(arr->items)</code> .
6	Por ultimo se aplica <code>arr->packed_block = block; arr->items = new_items; arr->packed_block_size = used; arr->capacity = arr->size;</code>

Así se logran varios beneficios como que el overhead por cada malloc pequeño (es decir, los metadatos internos del allocator y fragmentación) desaparece. Y también en las mediciones estimadas (lo que se pidió) típicamente se observa reducción del total solicitado.

F) Desempaquetamiento.

<code>students_unpack_all(StudentArray *arr)</code>	Para cada Student * que apunta dentro del bloque compacto, clona sus datos a malloc individuales llamando a student_clone y produce un arr->items con punteros a allocations individuales. Luego puede hacer uso de free(arr->packed_block). En este orden de ideas, esto tendría un "costo" $O(n)$ y requiere memoria temporal para las nuevas allocations. Esto se hace así para poder permitir modificaciones posteriores como serían añadir/borrar.
--	---

G) Estimación de uso de memoria.

<code>students_memory_usage(const StudentArray *arr)</code>	Devuelve una estimación de bytes "pedidos" por el programa, así, dependiendo de su estado puede actuar. Si está compactado: devuelve $\text{arr->packed_block_size} + \text{arr->capacity} * \text{sizeof(Student*)}$. Si no está compactado: suma para cada estudiante $\text{sizeof(Student)} + \text{strlen(first)+1} + \text{strlen(last)+1} + \text{n_grades} * \text{sizeof(double)}$ y añade $\text{capacity} * \text{sizeof(Student*)}$.
--	---

Limitación explícita: No usa malloc_usable_size ni incluye metadata interna del allocator. Es una medición consistente entre operaciones, lo que es útil para comparar antes/después. Por ejemplo, antes después del shrink-to-fit o de una compactación.

Por último, deben establecerse algunas aclaraciones, como las limitaciones prácticas, y es que, en ese sentido, se presentan algunas consideraciones clave:

- Las compactaciones son costosas si hay muchos cambios, es decir, cosas como compactar frecuentemente es caro a nivel de recursos.
- Realloc puede fallar: student_shrink_to_fit usa realloc y actualiza punteros solo si realloc devolvió no-NULL (por eso no rompemos el dato si falla).
- El desempaquetado también es costoso, ya que la implementación clona todos los registros; es útil y evita fallos, pero consume muchos más recursos.



3. CALCULO LAMBDA.

Se dio un código en lenguaje Haskell que dada una lista de n números, calcule el promedio (Es decir que sume todos los números de la lista y divida el resultado en la cantidad de números que hay en la lista) en ese orden de ideas, se nos solicitó escribir en notación de cálculo lambda la implementación del método que sale en la imagen anexa para calcular el promedio de números de una lista de tamaño n , por lo que mi planteamiento fue el siguiente:

$$FOLDER \equiv \lambda f. \lambda z. \lambda xs. xs \ f \ z$$

$$SUM \equiv \lambda xs. FOLDER (+) 0 \ xs$$

$$LEN \equiv \lambda xs. FOLDER (\lambda acc. acc + 1) 0 \ xs$$

$$PROMEDIO \equiv \lambda xs. \frac{(SUM \ xs)}{(LEN \ xs)}$$

$$PROMEDIO_SAFE \equiv \lambda xs. IF \ (isEmpty \ xs) \ THEN \ \perp \ ELSE \ \frac{(SUM \ xs)}{(LEN \ xs)}$$

Definiendo que es cada cosa, tenemos que:

- **FOLDER:** fold derecho. Aplica la función f acumulando desde la derecha sobre la lista xs empezando con el acumulador z
- **SUM:** suma todos los elementos de la lista xs usando FOLDER con operador $+$ y acumulador inicial 0 .
- **LEN:** cuenta el número de elementos de xs En cada paso ignora el elemento $()$ y suma 1 al acumulador.
- **PROMEDIO:** calcula la media aritmética dividiendo la suma entre la longitud (asumiendo $LEN_{xs} \neq 0$).
- **PROMEDIO_SAFE:** versión segura que evita dividir por cero. Si la lista está vacía devuelve $\perp$ (o el valor/acción que el profesor especifique).